

ArgusML: Complete Team Presentation Script & Viva Defense Guide

5-Member Project Distribution: Aakash (Lead), Krishna, Sanskar, Ghanshyam, Hari | **Total Unit Tests:** 17/17 Passed

1. Team Distribution & Module Ownership Matrix

This matrix outlines the exact technical ownership, data structures, algorithms, and source files for each of the 5 team members:

Team Member	Module & Role	DSA Concepts Implemented	Time / Space Complexity	Code Files Executed
Aakash (Project Lead)	Dynamic 4-Layer DAG & Reverse-BFS Root Cause Engine	- Directed Acyclic Graph (DAG) - Reverse-BFS Upstream Attribution - Forward-BFS Blast Radius - Multi-Factor RCS Scoring	- Reverse-BFS: $O(V + E)$ - Forward-BFS: $O(V + E)$ - Space: $O(V + E)$	dsa/graph.py rca/engine.py orchestrator.py
Krishna (Co-Lead)	Ingestion Queue, Sliding Window & Statistical Drift	- Circular FIFO EventQueue - Double-Ended Queue (Deque) 2-Sample Kolmogorov-Smirnov - Population Stability Index	- Queue Enqueue: $O(1)$ - Deque Evict: $O(1)$ - KS Test: $O(N \log N)$ - PSI: $O(N + B)$	dsa/queue.py dsa/deque.py drift/detector.py
Sanskar (Analytical Lead)	Incident Max-Heap & 1-Click Retraining Validation Gate	- Array-backed Binary Max-Heap - Custom <code>_sift_up</code> / <code>_sift_down</code> - Closed-Loop Validation Gate - Automated Version Promotion	- Push Incident: $O(\log N)$ - Pop Max: $O(\log N)$ - Peek Top Alert: $O(1)$ - Retrain Check: $O(S)$	dsa/heap.py simulator/universal_runner.py orchestrator.py
Ghanshyam (Core Systems)	In-Memory Model Registry & Multi-Model Traffic Simulator	- In-Memory Hash Map Store - Empirical Moments Baseline - Background Traffic Synthesizer - Controlled Covariate Drift	- Baseline Lookup: $O(1)$ - Model Switch: $O(1)$ - Traffic Stream: $O(K)$	dsa/registry.py simulator/traffic_simulator.py simulator/universal_runner.py
Hari (Observability & UI)	Session History Drawer, Post-Mortems & Interactive UI	- Chronological Audit History 1-Click DAG Snapshot Replay - Markdown Report Synthesizer - Canvas 2D Graph Engine	- Fast Prepend: $O(1)$ - Snapshot Lookup: $O(1)$ - Report Export: $O(R)$	dsa/history.py app/main.py frontend/js/app.js frontend/js/graph_renderer.js

2. Slide-by-Slide Word-for-Word Presentation Script

Follow this chronological speaking sequence during the group presentation. Each speaker covers their specific slides and hands over cleanly to the next speaker:

■ Slide 1 & 2: Introduction & Team Division (Speaker: AAKASH)

Exact Words to Speak:

*"Respected professors and panel members, good morning. Today, our team is presenting **ArgusML** — a Universal Production AI Observability and Graph-Powered Root Cause Diagnostic Platform built upon 5 foundational Data Structures and Algorithms.*

In today's presentation, we have divided the technical modules cleanly among 5 team members:

- **I (Aakash)** am leading the system architecture, dynamic 4-layer DAG topology modeling, and Reverse-BFS root cause attribution.
- **Krishna** will present the high-throughput circular FIFO ingestion queue, sliding window deque, and Kolmogorov-Smirnov statistical drift engine.
- **Sanskar** will explain the binary max-heap priority queue and the closed-loop automated model retraining validation gate.
- **Ghanshyam** will detail the in-memory hash map model registry and universal multi-model traffic simulation engine.
- **Hari** will showcase our session history audit drawer, 1-click historical snapshot replay, and downloadable markdown post-mortem generator.

Let us begin with the core problem that motivated this project."

■ Slide 3 & 4: Problem Statement, Novelty & DSA Mapping (Speaker: AAKASH)

"Unlike traditional web servers that throw 500 internal errors when they break, machine learning models fail silently. Real-world consumer behavior shifts, causing data distributions to drift, while the model continues making confident, incorrect predictions. When a model's accuracy drops from 95% to 65%, data engineering teams waste days manually tracing through dozens of multi-table ETL pipelines to isolate which specific feature was corrupted.

ArgusML solves this by implementing an entirely in-memory, pure DSA architecture with zero third-party vendor bloat. As shown in our DSA Mapping table on Slide 4, every system component maps directly to optimal theoretical complexity:

1. Circular FIFO Buffer for $O(1)$ ingestion without memory overflow.
2. Double-Ended Queue for amortized $O(1)$ rolling performance metrics.
3. In-Memory Hash Map for $O(1)$ model and baseline empirical distribution lookups.
4. Binary Max-Heap for $O(\log N)$ composite severity prioritization.
5. Dynamic Directed Acyclic Graph (DAG) for $O(V+E)$ reverse and forward BFS traversals.

I will now hand over to Krishna to explain how live predictions enter the system and how statistical drift is detected."

■ Slide 5 & 7: Ingestion Queue, Deque Window & Drift Analytics (Speaker: KRISHNA)

"Thank you, Aakash. I will now explain how ArgusML continuously watches streaming inferences and detects distribution shifts.

When client applications send inference payloads to `POST /api/ingest`, they enter our **Circular FIFO EventQueue** (`queue.py`). We implemented modulo pointer arithmetic so that enqueueing and dequeuing operate strictly in $O(1)$ time with a fixed capacity of 5,000 events. If an extreme traffic spike occurs, oldest unconsumed records are evicted gracefully, completely preventing memory leaks.

Next, a background stream processor dequeues batches into our **MetricSlidingWindow** (`deque.py`), which is an in-memory double-ended queue holding recent $W = 400$ predictions. This allows us to calculate rolling accuracy, precision, recall, and P99 latency in amortized $O(1)$ time.

Every 25 processed events, our **DriftEngine** (`detector.py`) executes two statistical tests against the baseline distributions stored in our Hash Map:

1. **Two-Sample Kolmogorov-Smirnov (KS) Test:** We compute the maximum vertical divergence D between the baseline empirical cumulative distribution function (eCDF) and the sliding window in $O(N \log N)$ time. If $p < 0.05$, we flag significant covariate drift.
2. **Population Stability Index (PSI):** We bucket feature values across quantile bins in $O(N + B)$ time. A $PSI \geq 0.25$ indicates critical distribution shift.

Once drift is detected, Aakash's DAG engine is triggered to isolate the root cause."

■ Slide 6: Dynamic DAG & Root Cause Confidence Scoring (Speaker: AAKASH)

"Once statistical drift is flagged, our **DependencyGraph** (`graph.py`) performs automated root cause attribution.

The DAG dynamically builds a 4-layer topology:

Upstream Pipelines → Inferred Features → Active Model → Downstream Services.

We execute two Breadth-First Search traversals, both operating in $O(V + E)$ linear time:

1. **Reverse-BFS Upstream Attribution:** Starting at the degraded Model node, we traverse reverse adjacency lists upstream to find which specific feature node and upstream ingestion pipeline originated the corruption.
2. **Forward-BFS Downstream Blast Radius:** We traverse forward adjacency lists downstream from the model node to identify every production consumer API and reporting service impacted.

To rank candidate causes with mathematical rigor, I designed the **Root Cause Confidence Score (RCS)** formula:

$RCS = (0.45 * DriftEvidence) + (0.30 * AccuracyDrop) + (0.15 * TopologyProximity) + (0.10 * BlastImpact)$.

This pinpoints the true culprit feature with over 85% to 95% confidence.

I will now hand over to Sanskar to explain how incidents are prioritized in the Max-Heap and how automated retraining cures the model."

■ Slide 8: Priority Max-Heap & 1-Click Retraining Validation Gate (Speaker: SANSKAR)

"Thank you, Aakash. I will now explain our **AlertMaxHeap** and the **Closed-Loop Automated Model Retraining Pipeline**.

In production environments monitoring multiple models, alerts must be prioritized so SREs tackle the most dangerous failures first. We implemented a pure, array-backed **Binary Max-Heap** (`heap.py`) using pure `_sift_up` and `_sift_down` pointer swaps. Pushing a new incident and extracting the maximum severity incident both run in strict $O(\log N)$ time, while peeking the top incident is $O(1)$. Incidents are ranked by composite severity taking into account accuracy drop, drift magnitude, and blast radius.

When an incident occurs, our **Automated Retraining Engine** (`orchestrator.py`) allows 1-click remediation with a strict **4-Step Validation Gate**:

1. It collects recent sliding window observations and fits a candidate model to the drifted distribution.
2. It evaluates 4 validation criteria: Data sufficiency, Accuracy $\geq$ SLA target, Latency $\leq$ SLA threshold, and positive R^2 score.
3. If validation passes, the candidate is promoted to production, the version is bumped (e.g. v1.0.0 $\rightarrow$ v1.1.0), Hash Map baselines are refreshed, active Max-Heap alerts are resolved, and DAG node health resets to HEALTHY.

I will now pass to Ghanshyam to explain our Model Registry and Traffic Simulator."

■ Slide 9: Model Registry & Universal Traffic Simulator (Speaker: GHANSHYAM)

"Thank you, Sanskar. I will cover the **ModelRegistry** and our **Universal Traffic Simulation Engine**.

The **ModelRegistry** (`registry.py`) serves as our central in-memory Hash Map baseline store. It provides $O(1)$ constant-time lookup for model metadata, SLA thresholds, and empirical baseline distribution moments (mean, variance, min, max, and quantiles). It universally supports both **Classification models** (tracking Accuracy and F1) and **Regression models** (tracking MAE, RMSE, and R^2).

To enable comprehensive testing without external hardware, I built the **UniversalTrafficSimulator** (`traffic_simulator.py`). It runs a background streaming thread pushing realistic predictions with Gaussian noise into the EventQueue at 20 events per second. It features controlled covariate shift injection — allowing users to dynamically multiply any feature's distribution by 3.5x or spike latency by +250ms with a single click.

Furthermore, our CSV upload parser automatically extracts numerical columns, fits reference models, registers baselines in HashMap, and builds dynamic DAG topologies on the fly.

I now hand over to Hari to present our session audit history, post-mortem generator, and user interface."

■ Slide 10, 11 & 12: Session History, Post-Mortems & Live Demo (Speaker: HARI)

*"Thank you, Ghanshyam. I will present our **Testing & Observability Session History Drawer**, **Incident Post-Mortem Exporter**, and the front-end dashboard.*

*In production, auditability is essential. We implemented **ModelTestingHistory** (`history.py`), which maintains a thread-safe chronological record of all model registrations, live drift detections, retraining runs, and switches. New session snapshots are prepended in $O(1)$ time. With our **1-Click Historical Snapshot Replay**, clicking any card in the drawer immediately loads the exact DAG topology canvas, statistical indicators (KS/PSI), and blast radius captured during that test run.*

*Our **Incident Post-Mortem Generator** automatically exports a comprehensive Markdown report documenting the executive summary, primary culprit feature, candidate RCS confidence rankings, impacted downstream services, and remediation logs.*

Finally, our web dashboard is built with clean Vanilla HTML5, CSS3, and an interactive Canvas 2D DAG renderer with pulsating node halos. All 17 unit tests in our Pytest suite pass with 100% success.

We are now ready for the live demonstration and panel questions."

3. Individual Code Breakdown & Files Executed

Use this section during technical viva when professors ask: *'Which specific files and code functions did you write and execute?'*

Aakash (Lead):

- **backend/app/core/dsa/graph.py**: Implemented `DependencyGraph` class with `forward_adj` and `reverse_adj` adjacency lists. Built `build_topology_for_model()`, `get_upstream_nodes(model_id)` via Reverse-BFS, and `get_downstream_nodes(model_id)` via Forward-BFS.
- **backend/app/core/rca/engine.py**: Built `RootCauseAnalysisEngine` and `diagnose_model()`. Formulated the 4-factor Root Cause Confidence Score (RCS) weighting drift evidence, accuracy drop, graph proximity, and blast radius.
- **backend/app/core/orchestrator.py**: Architected `ArgusSystem` singleton orchestrator coordinating `EventQueue`, `MetricSlidingWindow`, `DriftEngine`, `AlertMaxHeap`, and `ModelRegistry`.

Krishna:

- **backend/app/core/dsa/queue.py**: Built `EventQueue` circular buffer with head/tail pointers, modulo wrapping `self.tail = (self.tail + 1) % self.capacity`, and `enqueue()/dequeue()` $O(1)$ operations.
- **backend/app/core/dsa/deque.py**: Built `MetricSlidingWindow` using `collections.deque` with fixed `maxlen`. Built `get_performance_metrics()` and `get_latency_metrics()` for rolling P99 latency.
- **backend/app/core/drift/detector.py**: Built `DriftEngine` implementing `compute_ks_test()` using `scipy.stats.ks_2samp` and `compute_psi()` with quantile binning.

Sanskar:

- **backend/app/core/dsa/heap.py**: Built `AlertMaxHeap` with pure `_sift_up` and `_sift_down` array methods. Built `push()`, `pop_max()`, `peek()`, and `resolve(alert_id)` for $O(\log N)$ incident ranking.

- **backend/app/core/simulator/universal_runner.py**: Built `evaluate_validation_gate()` implementing the 4-step validation criteria (data sufficiency, accuracy recovery, latency check, non-negative R2).
- **backend/app/core/orchestrator.py**: Built `retrain_active_model()` executing candidate fitting, version bumping (v1.0.0 -> v1.1.0), Max-Heap alert clearing, and node health resets.

Ghanshyam:

- **backend/app/core/dsa/registry.py**: Built `ModelRegistry` and `ModelMetadata` classes. Implemented `register_model()`, `get_model()`, `register_feature_baseline()`, and moments calculation in $O(1)$ `HashMap`.
- **backend/app/core/simulator/traffic_simulator.py**: Built `UniversalTrafficSimulator` running background thread `_stream_loop()` generating synthetic inferences with Gaussian noise and latency spikes.
- **backend/app/core/simulator/universal_runner.py**: Built `UniversalModel` wrapper supporting both Classification and Regression scikit-learn models.

Hari:

- **backend/app/core/dsa/history.py**: Built `ModelTestingHistory` and `TestSessionRecord` dataclass. Implemented `record_session()` with in-place deduplication and `get_history()`.
- **backend/app/main.py**: Built FastAPI REST API exposing `/api/dashboard`, `/api/history`, `/api/history/{id}`, `/api/models/upload-csv`, and `/api/alerts/{id}/report`.
- **frontend/js/app.js & graph_renderer.js**: Built Canvas 2D interactive DAG topology renderer, session history drawer renderer, snapshot inspection modal, and toast notification engine.

4. Expected Professor Viva Q&A; Guide (High-Scoring Answers)

Q1 (For Aakash): Why did you choose Reverse-BFS over Dijkstra or DFS for Root Cause Analysis?

Answer: In a dependency DAG where all dependency relationships are discrete unweighted links between upstream ETL pipelines, features, and model nodes, Breadth-First Search (BFS) explores dependencies layer by layer in strictly optimal $O(V + E)$ time. Dijkstra would add unnecessary $O(E \log V)$ heap overhead for unweighted edges, while DFS can explore deep non-critical paths first. Reverse-BFS guarantees finding the nearest upstream corrupted feature with shortest topological proximity first.

Q2 (For Krishna): How does the Circular FIFO Queue prevent memory overflow during traffic spikes?

Answer: Our `EventQueue` allocates a fixed circular buffer of capacity $C = 5000$ using modulo pointer wrapping: `self.tail = (self.tail + 1) % self.capacity`. When the queue is full (`size == capacity`), new events overwrite the oldest unconsumed entry and increment a `dropped_events` counter. This ensures $O(1)$ constant-time ingestion while guaranteeing memory usage never exceeds the pre-allocated buffer size.

Q3 (For Sanskar): Why is a Binary Max-Heap superior to an array or sorted list for incident management?

Answer: An unsorted array requires $O(N)$ linear time to search for the highest-severity alert, which is too slow in high-throughput systems. A sorted array requires $O(N)$ time on every new alert insertion. A Binary Max-Heap provides the optimal theoretical balance: $O(\log N)$ insertion via `_sift_up` and $O(\log N)$ extraction of the critical alert via `_sift_down`, while peeking the active top alert is instantaneous $O(1)$.

Q4 (For Ghanshyam): How does ModelRegistry achieve $O(1)$ baseline retrieval, and how does it support multiple models?

Answer: ModelRegistry is backed by Python's in-memory Hash Map (dictionary) indexed by unique `model_id` strings. Each ModelMetadata record maintains an inner hash map `feature_baselines` mapping feature names to EmpiricalBaseline objects containing baseline sample arrays and statistical moments. Hash map bucket indexing provides $O(1)$ average-time retrieval regardless of how many models are registered in the platform.

Q5 (For Hari): How does the Session History Drawer capture and replay historical DAG snapshots?

Answer: When a drift incident, model switch, or retraining occurs, ModelTestingHistory captures a deep-copy snapshot of the graph adjacency list and node health states inside `TestSessionRecord.graph_snapshot`. When a user clicks a historical record in the UI, our frontend GraphRenderer reads that snapshot and replays the exact canvas topology and red-highlighted culprit nodes without needing to roll back active live production telemetry.